

Idris Sadiq

Senior Software Engineer

idrissadiqdev@outlook.com +1 (424) 458-6548 | Los Angeles, California, United States

LinkedIn: [linkedin.com/in/idris-sadiq-958b453a6](https://www.linkedin.com/in/idris-sadiq-958b453a6)

PROFESSIONAL SUMMARY

Senior Software Engineer with ~10 years building and operating Python + JavaScript platforms end-to-end, pairing React UI work with reliable backend APIs, data modeling, and production operations. Strong focus on scalable web products, performance tuning, and pragmatic engineering that turns ambiguous requirements into shippable increments with clear ownership. Experienced modernizing legacy frontends, designing REST/GraphQL services, building event-driven backends, and improving observability to reduce on-call noise. Comfortable partnering with product and non-engineering stakeholders, setting measurable outcomes, and delivering features safely through testing, CI/CD, and staged rollouts.

SKILLS

Programming: Python, JavaScript (ES6+), TypeScript, Node.js, SQL, Bash, Go (basic scripting for tooling), HTML5, CSS3, OOP & SOLID, Functional programming patterns, Concurrency (async/await, queues, workers), Data structures & algorithms, Performance profiling & optimization

Python Backend: FastAPI, Django, Flask, Celery, SQLAlchemy, Alembic migrations, Pydantic, Uvicorn/Gunicorn deployment patterns, Async Python patterns, Background jobs & schedulers, Webhook processing & verification, OpenAPI/Swagger documentation, Rate limiting, retries, and backoff, Idempotency patterns for write endpoints, ETL scripting and data pipelines, Secure secret handling and config management

Frontend: React, Next.js, Redux, React Router, Formik, Vite, Webpack, Tailwind CSS, Material UI, Bootstrap, Storybook, Micro-frontend patterns, Accessibility (WCAG-minded UI), Performance profiling (Lighthouse, Web Vitals), SSR/SSG/ISR patterns, Design systems and component libraries, Browser performance (bundle splitting, memoization, caching)

Backend: Express.js, NestJS, REST API Design, GraphQL, WebSockets, Background Jobs, Event-Driven Architecture, Message queues (SQS-like patterns), Caching Strategies, API Versioning, Auth (JWT, OAuth 2.0), RBAC and permission modeling, File/Media Handling (S3-style workflows), Pagination, filtering, and sorting contracts, Error handling and consistent API envelopes

Databases: PostgreSQL, MySQL, MongoDB, Redis, Schema Design, Indexing Strategies, SQL Query Tuning, Data Modeling for Analytics, Migration workflows (zero/low-downtime), Read/write patterns and connection pooling, Transactional integrity and locking awareness

Cloud: AWS, S3, CloudFront, API Gateway, Lambda, ECS, SQS, SNS, EventBridge, CloudWatch, IAM, Secrets Management (SSM/Secrets Manager patterns), Cost-aware architecture decisions, Infrastructure hardening (least privilege, network controls)

DevOps: Docker, Kubernetes, Terraform, CI/CD Pipelines, GitHub Actions, Jenkins, Release Automation, Artifact Versioning & Promotion, Blue/Green and Canary rollouts, Environment parity and configuration hygiene, Runbooks and operational readiness

Observability: Structured Logging, Metrics & Dashboards, Distributed Tracing, Prometheus, Grafana, Datadog, SLO/SLI Design, Incident Response, Postmortems and follow-ups, Alert tuning and on-call noise reduction

Testing: pytest, Jest, React Testing Library, Cypress, Integration Testing, Contract Testing, Load/Performance Testing (k6-like patterns), Test pyramids and pragmatic coverage, Mocking, stubbing, and fixtures, CI test optimization and flaky test remediation

Architecture & Practices: System Design, Clean Architecture / Layered Services, Monolith-to-services decomposition, Secure-by-default engineering, Code Reviews & Mentorship, Incremental delivery and scoping, Documentation and runbooks, Stakeholder management and expectation setting, Risk management via staged rollouts and feature flags

EXPERIENCE

Senior Software Engineer

SumatoSoft | January 2021 - December 2025

- Led a Python + React delivery stream for a high-traffic customer workflow, converting ambiguous product asks into scoped milestones, measurable outcomes, and weekly releases that stakeholders could reliably validate end to end.
- Designed and shipped FastAPI services with clear versioning, consistent error contracts, and strict request/response validation, reducing integration churn for client teams that depended on stable APIs and predictable behaviors.
- Modernized a React + TypeScript codebase by introducing a component-driven architecture, strict linting/formatting gates, and a stable UI composition strategy that decreased regressions during frequent feature additions.
- Implemented robust idempotency and retry-safe write paths for order-like operations, preventing duplicate side effects under client retries and improving correctness during transient failures and partial downstream outages.
- Built background processing pipelines using Python workers for long-running tasks, separating user-facing latency from heavy processing while keeping traceability through correlation IDs and structured logging.
- Optimized database access patterns with indexing, query rewrites, and careful pagination contracts, improving response consistency under load and lowering operational stress during peak usage windows.
- Introduced observability improvements by standardizing log fields, adding service-level metrics, and tightening alert thresholds so on-call pages represented actionable incidents rather than noisy symptoms.
- Created deployment safeguards with staged rollouts and feature flags, allowing safe experimentation, quick reversals, and reduced blast radius when changes interacted unexpectedly with real production traffic.
- Partnered closely with product and QA to define acceptance criteria that mapped to testable behaviors, ensuring features shipped with fewer ambiguous edge cases and a clearer shared definition of “done.”
- Built internal tooling in Python to automate repetitive operational tasks, cutting manual support effort and enabling faster triage when customers reported issues that required data inspection or replay.
- Mentored engineers through code reviews focused on correctness, readability, and maintainability, while also coaching on incident follow-ups so fixes addressed root causes instead of repeatedly treating symptoms.
- Implemented authentication and authorization patterns across services, enforcing least-privilege access and consistent permission checks that reduced security risk without slowing down feature delivery cadence.
- Improved frontend performance by reducing unnecessary re-renders, trimming bundle weight via code splitting, and addressing key Web Vitals issues that impacted real user sessions on slower devices and networks.
- Aligned backend and frontend contracts with explicit API documentation and examples, reducing miscommunication between teams and making onboarding faster for new engineers joining the product area midstream.
- Owned production reliability by driving post-incident reviews into concrete backlog items, tracking them to completion, and steadily lowering recurring incident categories that previously consumed on-call time.

Senior Software Engineer

High Touch Technologies | June 2018 - December 2020

- Owned a React single-page application supporting complex operational workflows, refining state management and component boundaries so new features could ship quickly without brittle UI coupling or hidden side effects.
- Delivered backend endpoints and service logic with clear validation and consistent pagination/filtering semantics, improving downstream consumer reliability when integrating across multiple internal clients and teams.
- Implemented backend routines in Python for data-heavy processing needs, separating compute-intensive work from synchronous requests to keep user-facing latency stable even under bursty operational loads.
- Built reusable UI patterns and form abstractions that reduced duplicated code across similar screens, increasing development velocity while keeping behavior consistent for end users across the product surface.

- Added pragmatic automated tests around core workflows, targeting regression-prone areas and building confidence to refactor legacy modules without introducing subtle breakage into critical business paths.
- Collaborated with product leadership to translate operational pain points into prioritized engineering work, ensuring the roadmap reflected measurable improvements rather than vague, difficult-to-verify efforts.
- Improved API error handling and observability by standardizing response envelopes and logging context, making it easier for support and engineering to diagnose failures with less back-and-forth.
- Optimized database queries and tuned indexes for high-volume tables, reducing slow queries that impacted operational users during high concurrency periods and improving overall system responsiveness.
- Established code review norms emphasizing maintainability and clear ownership boundaries, which reduced rework cycles and helped the team converge on shared patterns for common technical problems.
- Built secure authentication flows and session handling, aligning client behavior with server enforcement to reduce edge-case authorization bugs that previously surfaced as confusing user-facing errors.
- Created lightweight internal scripts to validate data assumptions during migrations and feature rollouts, preventing production issues caused by unexpected historical data shapes and missing constraints.
- Helped stabilize release delivery by improving CI checks and tightening merge discipline, ensuring that integration issues were found earlier rather than late in the release window.
- Documented key workflows and operational runbooks, making it easier for engineers and support staff to respond consistently during incidents and reducing reliance on tribal knowledge.

Software Engineer

Centric Tech Inc. | September 2015 - May 2018

- Developed React-based UI features to progressively replace server-rendered pages, maintaining behavior parity while improving usability and reducing friction across multi-step user workflows.
- Implemented backend services and utilities with reliable input validation and predictable error handling, which reduced ambiguous failures and improved support outcomes for customer-facing modules.
- Designed SQL schemas and indexes for evolving product requirements, balancing normalization with performance so core workflows stayed responsive as data volume and feature complexity increased.
- Built reusable UI components and shared utilities to reduce duplication, enabling faster delivery of consistent experiences across multiple screens without repeated reinvention of common behaviors.
- Improved application reliability by adding targeted automated tests and fixing recurring production issues, focusing on the paths users depended on most rather than superficial coverage metrics.
- Partnered with stakeholders to clarify requirements and edge cases early, translating loosely defined requests into implementable tasks with explicit acceptance criteria and clear delivery sequencing.
- Introduced basic performance tuning across the stack by profiling slow endpoints and addressing bottlenecks, ensuring the product remained usable as concurrency and customer adoption grew.
- Maintained and improved deployment readiness by documenting configuration expectations and environment differences, reducing surprise failures when code moved between development, staging, and production.
- Implemented secure handling for user inputs and sensitive data flows, reinforcing server-side checks so the system behaved safely even when clients sent unexpected or malformed requests.
- Helped onboard teammates by sharing patterns, reviewing changes with context, and creating short guides for common workflows so the team could move faster without sacrificing code quality.
- Worked closely with QA to reproduce and resolve hard-to-trace defects, ensuring fixes addressed the root cause and included regression checks to prevent the same issue from resurfacing later.

Software Engineer Intern

Microsoft | September 2014 - August 2015

- Contributed to a web-facing product area by implementing React UI enhancements and improving client-side reliability, delivering changes through review gates and internal quality checks used by the team.
- Built small service endpoints and developer utilities with careful input validation and predictable outputs, enabling safer reuse across internal workflows without introducing fragile assumptions.
- Collaborated with engineers and PMs to break down tasks into clear deliverables, communicating progress early and adjusting scope when new constraints or edge cases were discovered during implementation.
- Improved debugging and troubleshooting efficiency by adding meaningful logs and reproducing issues in controlled environments, reducing time to identify root causes for customer-reported problems.

- Wrote and updated documentation for setup steps and common development tasks, lowering onboarding friction for others and reducing repetitive questions during active sprint execution.
- Participated in code reviews and incorporated feedback quickly, learning team conventions and strengthening the maintainability of changes so they could be safely extended by others later.
- Helped validate fixes with targeted manual checks and lightweight test updates, ensuring the delivered work improved user experience without causing regressions in nearby behaviors.

EDUCATION

Harvard University

Bachelor's degree in Computer Science

2010 - 2014